Νευροασαφής Έλεγχος

Αναφορά 3ης Άσκησης

Κόνιας Οδυσσέας Γεώργιος 0311916

Μέρος 1: Αλγόριθμοι Βελτιστοποίησης

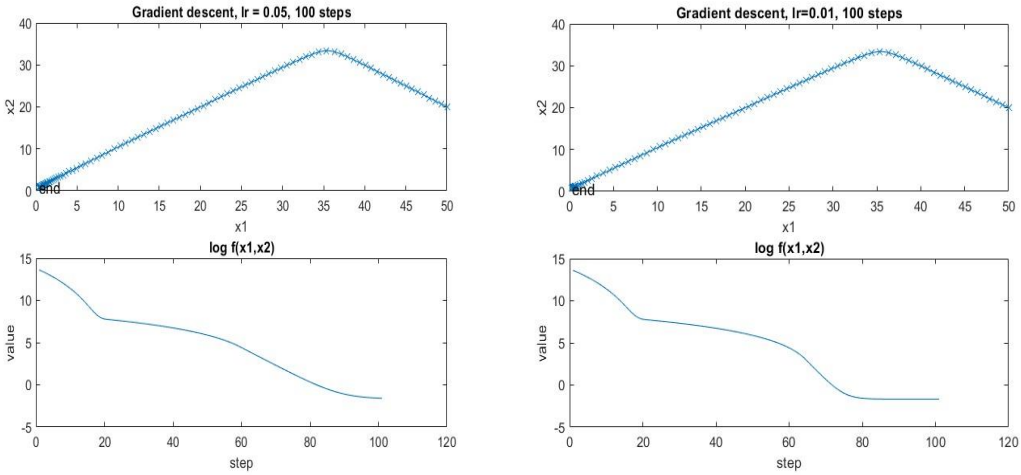
Θέμα 1

Υλοποιούμε τον αλγόριθμο gradient descent στο Matlab:

```
x0 = randn(2,1)*10;
x = x0;
%gradient descent
syms f(a,b);
f(a,b) = a^2+(b-1)^2+(a-b)^4;
x_plot =x;
f_plot = f(x(1),x(2));
%gradient descend
step_size = 0.01;
for i=1:30
    x1 = x(1);
    x2 = x(2);
    grad = [2*x1+4*(x1-x2)^3;2*(x2-1)+4*(x2-x1)^3];
    step =step_size*grad;
    if norm(step)>1
        step = step/norm(step);
    end
    x_new = x - step;
    x_plot =[x_plot,x_new];
    f_plot = [f_plot,f(x_new(1),x_new(2))];
    x= x_new;
end
subplot(2,1,1);
plot(x_plot(1,:),x_plot(2,:),'-x');
hold on
text(x(1),x(2),'end');
title('Gradient descent')
xlabel('x1')
ylabel('x2')
subplot(2,1,2);
plot(log(f_plot))
xlabel('step')
ylabel('value')
title('log f(x1,x2)')
```

Για ορισμένες αρχικές συνθήκες ο αλγόριθμος απαιτεί πολύ μικρό learning rate για να λειτουργήσει αυτό συμβαίνει γιατι ειδικά στην αρχή του αλγορίθμου οι παράγωγος είναι πολύ μεγάλη. Για αυτό προσθέτουμε την κανονικοποίση του βήματος αν αυτό έχει μέτρο μεγαλύτερο του 1. Με αυτή την προσθήκη μπορούμε να χρησιμοπιούμε μεγαλύτερο learning rate.

Παράδειγμα για αρχική θέση [50,20], 100 βήματα.



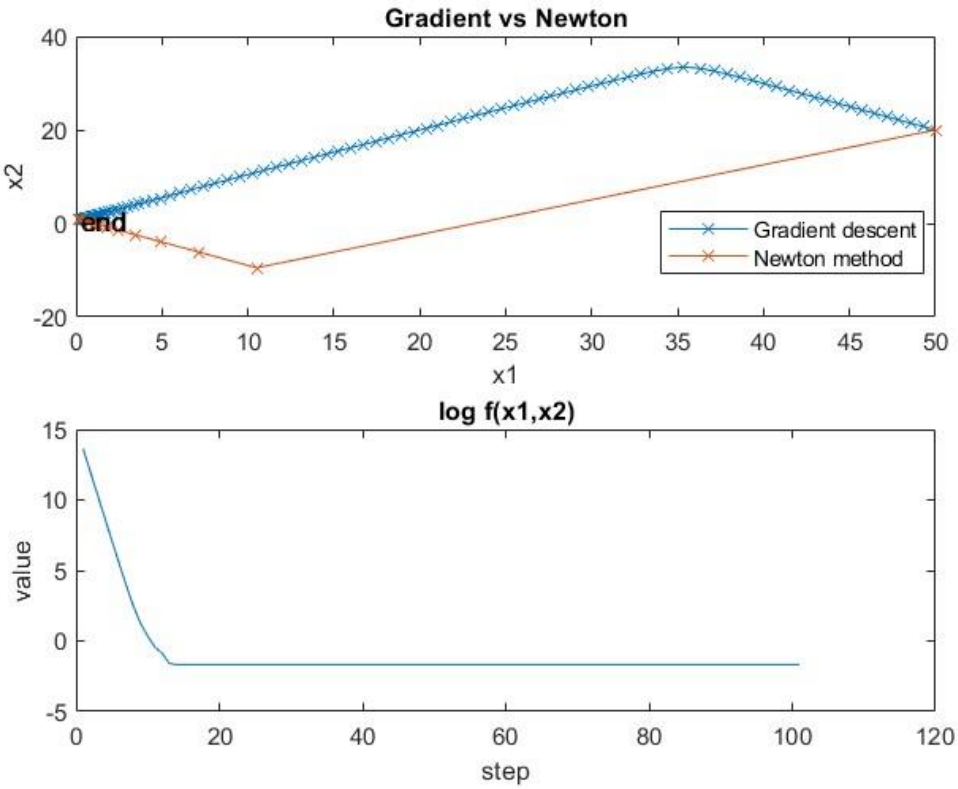
Για μεγαλύτερο Learning rate το κόστος πέφτει πιο γρήγορα σε ορισμένα μόνο βήματα του αλγορίθμου. Ακόμα χώρις την κανονικοποίηση η αύξηση του learning rate μπορεί να έχει αποτέλεσμα ο αλγόριθμος να μην βρεί ελάχιστο.

Αρχικό σημείο [30,20], 30 βήματα

Υλοποιούμε τον αλγόριθμο Newton:

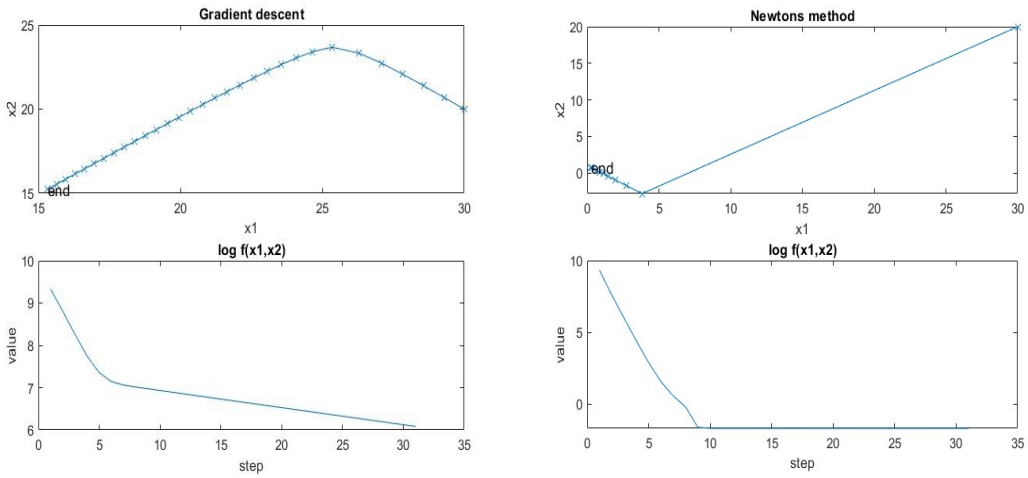
```
x0 = randn(2,1)*10;
%x0=[30;20]
x = x0
%gradient descent
syms f(a,b);
f(a,b) = a^2+(b-1)^2+(a-b)^4;


x_plot =x;
f_plot = f(x(1),x(2));

%newtons method
for i=1:30
    x1 = x(1);
    x2 = x(2);
    h1 = (12*(x1-x2)^2+2)/(48*x1^2-96*x1*x2+48*x2^2+4);
    h2 = (3*(x1-x2)^2)/(12*x1^2-24*x1*x2+12*x2^2+1);
    inv_Hessian = [h1 h2;h2 h1];
    grad = [2*x1+4*(x1-x2)^3;2*(x2-1)+4*(x2-x1)^3];
    step = inv_Hessian*grad;
    x_new = x - step;
    x_plot =[x_plot,x_new];
    f_plot = [f_plot,f(x_new(1),x_new(2))];
    x= x_new;
end
```

Παράδειγμα Newton για αρχική θέση [50, 20] μαζί με τα βήματα του gradient descent.



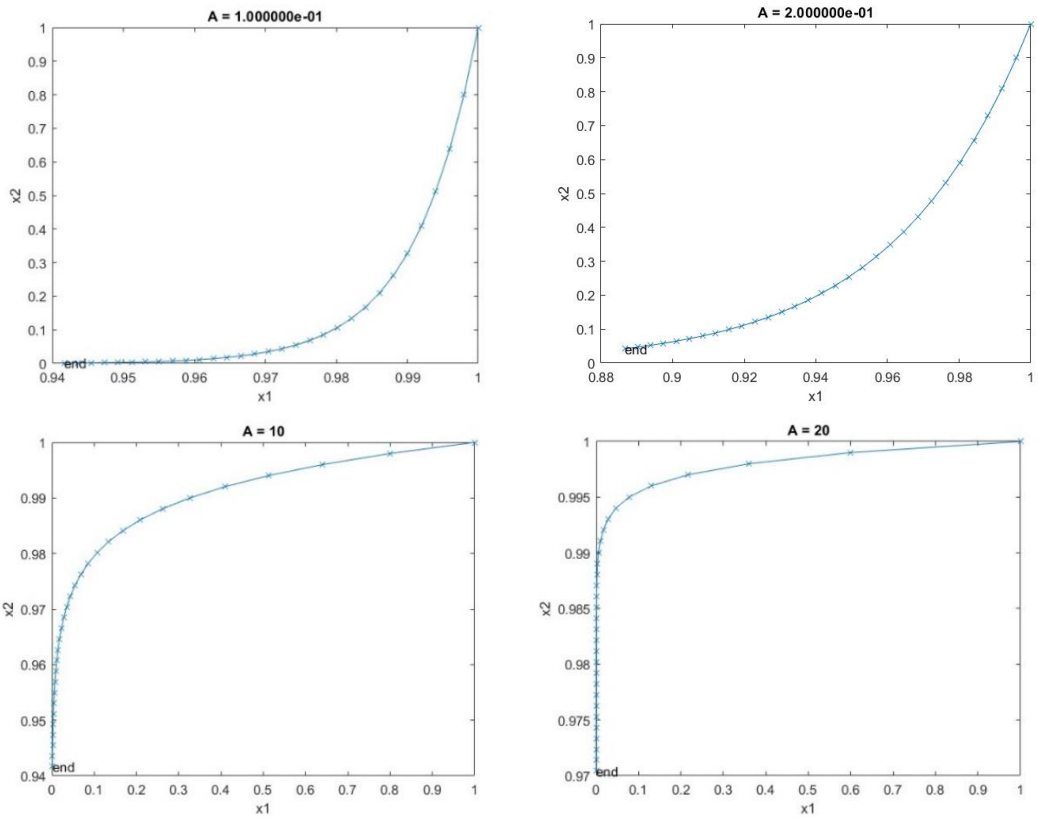
Βλέπουμε ότι ο αλγόριθμος Newton φτάνει στο ελάχιστο πολύ πιο γρήγορά. Η πτώση του κόστους είναι εκθετική.

Αρχική θέση [30,20]:



Θέμα 2

Προσπαθούμε να ελαχιστοποιήσουμε την συνάρτηση $f(x1, x2) = Ax_1^2 + \frac{1}{A}x2^2$. Το gradient είναι $\frac{\theta f}{\theta x1} = 2Ax_1, \frac{\theta f}{\theta x2} = \frac{2}{A}x_2$. Για τιμές A διάφορες του 1 η παράγωγος αυξάνεται σημαντικά στην μία διέυθυνση σε σχέση με την άλλη. Δοκιμάζουμε το gradient descent στο matlab (αντίστοιχα με το Θέμα 1):



\

Το ελάχιστο είναι το σημείο (0, 0). Για τιμές A<1 το x2 φτάνει στο 0 ενώ το x1 μειώνεται πάρα πολύ αργά. Για τιμές A>1 το x1 φτάνει στο 0 ενώ το x2 μειώνεται πολύ αργά.

Μπορούμε να διαπιστώσουμε το πρόβλημα εξετάζοντας και το condition number του Hessian πίνακα. Το condition number μας δίνει μια ένδειξη για την κλίμακα της διαφοράς των ιδιοτιμών ενός πίνακα. Εάν για τον Hessian πίνακα το condition number είναι μεγάλο αυτό σημαίνει ότι η συνάρτηση μας είναι πολύ πιο κυρτή σε μια διεύθυνση σε σχέση με μια άλλη. Αυτό εμποδίζει το gradient descent από το να λειτουργήσει αφού η παράγωγοι σε κάθε διεύθυνση είναι διαφορετικής τάξης μεγέθους.

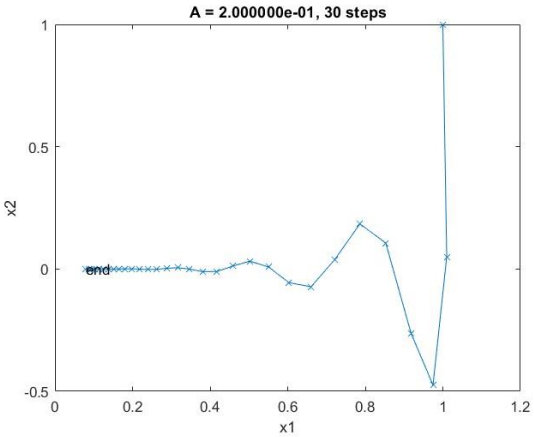
Ο hessian πίνακας: $\begin{bmatrix} 2A & 0 \\ 0 & \frac{2}{A} \end{bmatrix}$. Οι ιδιοτιμές είναι λ1 = 2A, λ2 = 2/A. Το Condition number για A>1 κ =A^2, για A<1 κ =1/A^2. Άρα για A διάφορο του 1 το condition number είναι μεγάλο.

Μία λύση στο πρόβλημα μας είναι η μέθοδος της ορμής. Το gradient descent μετατρέπεται σε: $x_{k+1} = x_k + v_{k+1}, \mathrm{v}_{\mathrm{k+1}} = a_1 v_k - a_2 \nabla f(x_k)$. Προσθέτουμε δηλαδή στο gradient descent τον όρο vk ο οποίος είναι ουσιαστικά ένας μεταβλητός μέσος όρος.

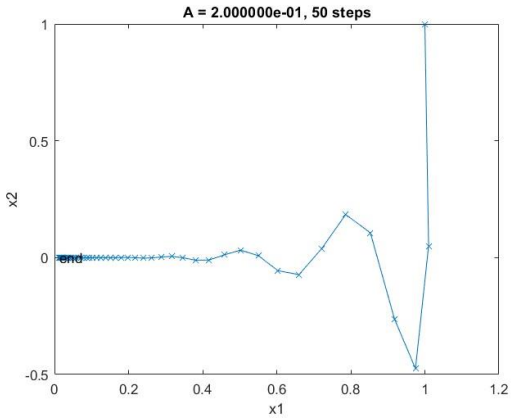
Ο κώδικας για την μέθοδο της ορμής:

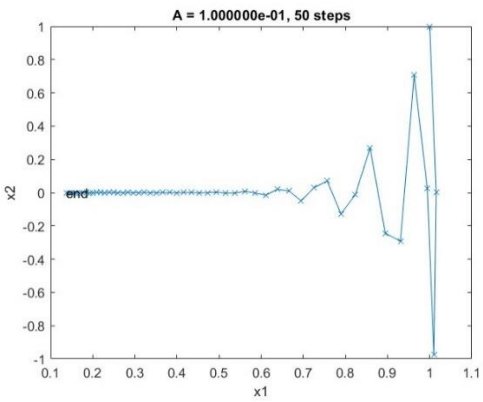
```
a1 = 0.2;
a2 = 0.2;
v = 0.1;
for i=1:30
    x1 = x(1);
    x2 = x(2);
    grad = [2*A*x1; (2/A)*x2];
    v_new = a1*v-a2*grad;
    step =-v_new;
    if norm(step)>1
        step = step/norm(step);
    end
    x_new = x - step;
    x_plot =[x_plot,x_new];
    f_plot = [f_plot,f(x_new(1),x_new(2))];
    x= x_new;
    v=v_new;
end
```

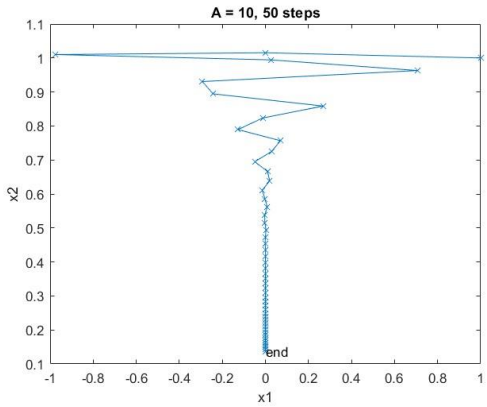
Αποτελέσματα:

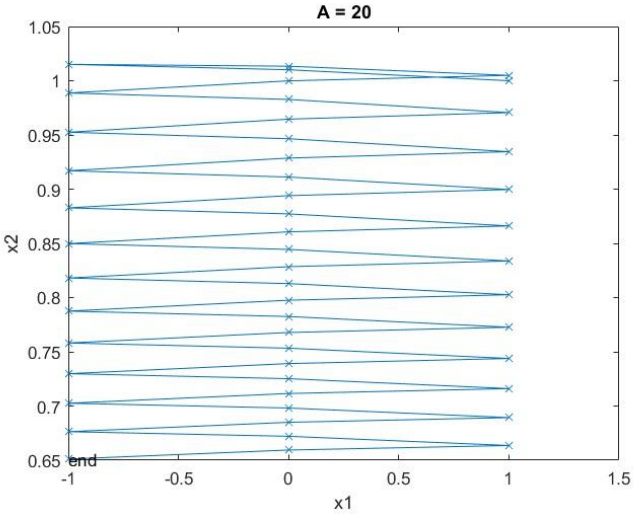


Το gradient descent πλησιάζει το ελάχιστο αρκεί να αυξήσουμε τον αριθμό των βημάτων:

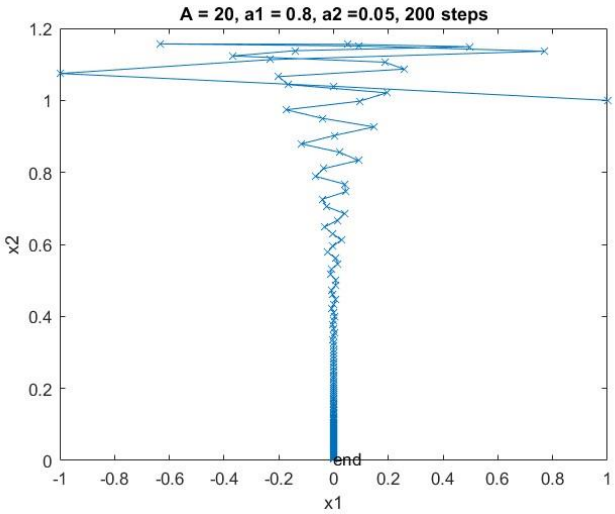








Για A=20 παρατηρούμε ότι το learning rate a2 είναι μεγάλο. Δοκιμάζουμε διαφορετικές τιμές:



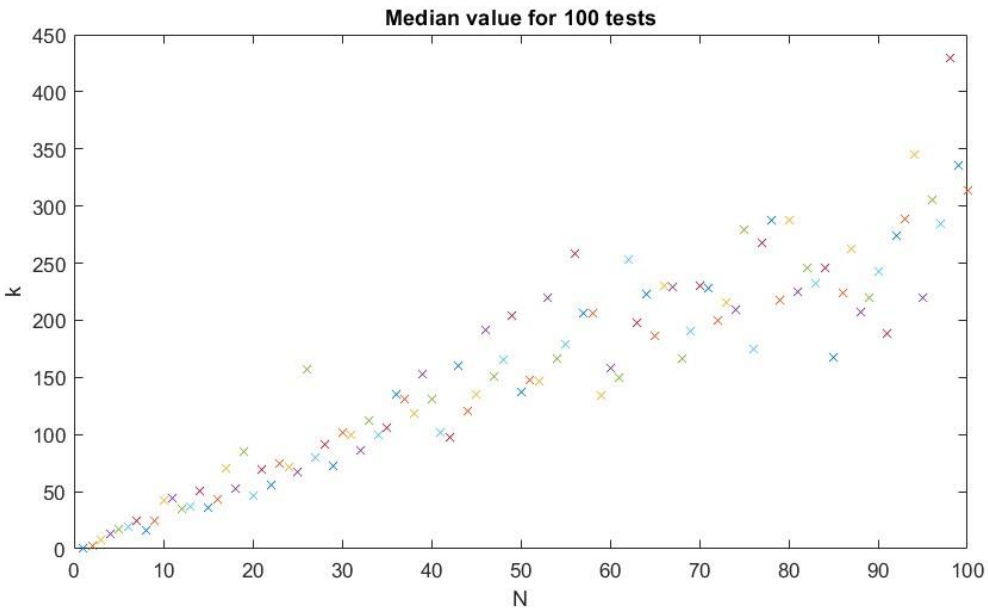
Θέμα 3

Στο Maltab τρέχουμε τον ακόλουθο κώδικα:

```
for N=1:100
    k_values = [];
    for i=1:100

        A = randn(N,N);
        Q = A*A;
        eigenvalues = eig(Q);
        k =  abs(max(eigenvalues))/abs(min(eigenvalues));
        k_values = [k_values, k];
    end
    median_value = median(k_values);
    plot(N,median_value, 'x-');
    hold on
end
```

Για κάθε N υπολογίζουμε το condition number για 100 τυχαίους πίνακες διάστασης N. Κρατάμε την διάμεσο για να έχουμε μια ένδειξη της τάξης μεγέθους. (Η διάμεσος δούλεψε καλύτερα από τον μέσο όρο αφού δεν επηρεάζεται από τις ακραίες τιμές).

Κάνουμε plot τα αποτελέσματα:



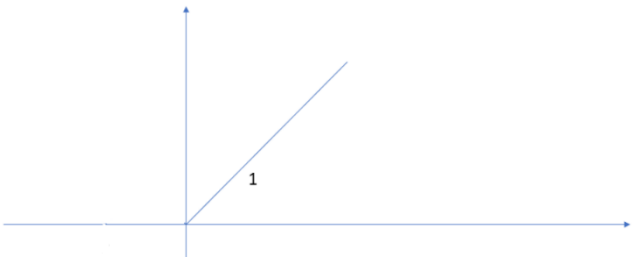
Βλέπουμε ότι υπάρχει ξεκάθαρη συσχέτιση μεταξύ του condition number και της διάστασης του πίνακα. Το αποτέλεσμα είναι λογικό αφού όσο μεγαλύτερη η διάσταση τόσο μεγαλύτερη πιθανότητα δύο κατευθύνσεις να έχουν πολύ διαφορετικές ιδιοτιμές.

Μερος 2: Απλά Νευρωνικά Δίκτυα

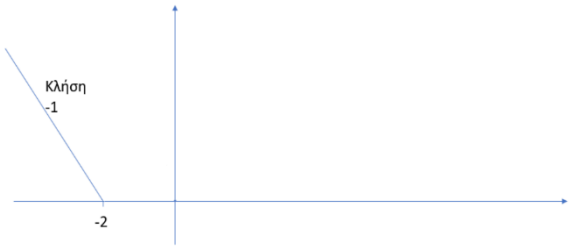
Θέμα 1.

Χωρίζουμε την συνάρτηση σε κομμάτια και την αναπαριστούμε με την συνάρτηση ReLU.

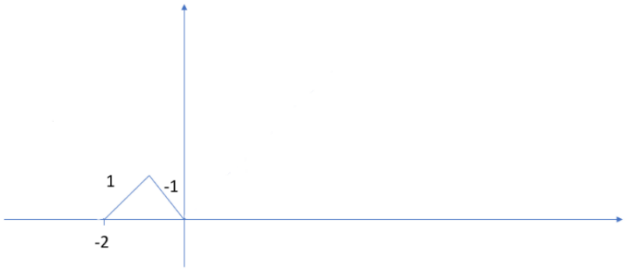
$$y_1 = ReLU(x)$$



$$y_2 = ReLU(-x-2)$$



$$y_3 = \ ReLU(-x) - 2ReLU(-x-1) + ReLU(-x-2)$$



Συνολικά έχουμε:

$$y = ReLU(x) + 2ReLU(-x-2) + ReLU(-x) + 2ReLU(-x-1)$$

Η έξοδος του νευρωνικού θα είναι:

$$y_{nn} \ = \ W2ReLU(W1x + b1) + b2$$

όπου W_1 , W_2 τα βάρη και b_1, b_2 τα biases του κρυφού στρώματος και του στρώματος εξόδου αντίστοιχα. Ορίζουμε τα διανύσματα κατάλληλα ώστε να έχουμε y_{nn} = y:

$W_1 = [1 - 1 - 1 - 1\ ]$

$W_2 = [1\ 2\ 1\ 2]$

$b_1 = [0 - 2\ 0 - 1]$

$b_2 = [0\ 0\ 0\ 0]$

Η υλοποίηση του νευρωνικού σε python με χρήση του pytorch

```
import torch
import torch.nn as nn
import torch.optim as optim

# Define the neural network
class CustomNN(nn.Module):
    def __init__(self):
        super(CustomNN, self).__init__()
        # Initialize weights and biases for the hidden layer
        self.hidden_weights = torch.tensor([[1.0], [-1.0], [-1.0], [-1.0]],
requires_grad=True)
        self.hidden_biases = torch.tensor([0.0, -2.0, 0.0, -1.0], requires_grad=True)

        # Initialize weights and biases for the output layer
        self.output_weights = torch.tensor([1.0, 2.0, 1.0, 2.0], requires_grad=True)
        self.output_biases = torch.tensor([0.0, 0.0, 0.0, 0.0], requires_grad=True)

    def forward(self, x):
        # Hidden layer operations
        x = x * self.hidden_weights.t() + self.hidden_biases
        x = torch.relu(x)

        # Output layer operations
        x = x * self.output_weights + self.output_biases
        x = x.sum(dim=1, keepdim=True)

        return x
```

Θέμα 2

Το αρχικό μοντέλο:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(
from_logits=True),
              metrics=['accuracy'])
```

```
Epoch 10/10
1875/1875 [==============================] - 9s 5ms/step - loss:
0.2401 - accuracy: 0.9111

Test accuracy: 0.8791000247001648
```

Δοκιμάζουμε διαφορετικό optimizer:

SGD:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.01)
Epoch 10/10
1875/1875 [==============================] - 6s 3ms/step - loss:
0.3717 - accuracy: 0.8703

Test accuracy: 0.8547999858856201
```

RMSprop:

```
optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.001)
Epoch 10/10
1875/1875 [==============================] - 6s 3ms/step - loss:
0.2703 - accuracy: 0.9058

Test accuracy: 0.8743000030517578
```

Adadelta:

```
optimizer = tf.keras.optimizers.Adadelta(learning_rate=1.0)
Epoch 10/10
1875/1875 [==============================] - 9s 5ms/step - loss:
0.2502 - accuracy: 0.9090

Test accuracy: 0.8858000040054321
```

Ο optimizer adadelta πετυχαίνει μικρή βελτίωση στην ακρίβεια του μοντέλου στο test set

Δοκιμάζουμε διαφορετικά μεγέθη στρωμάτων (με optimizer = ‘Adam’)

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dense(10)
])
Epoch 10/10
1875/1875 [==============================] - 5s 3ms/step - loss:
0.3032 - accuracy: 0.8897

Test accuracy: 0.8679999709129333
```

Η ακρίβεια του μοντέλου πέφτει, μειώνεται ο χρόνος εκπαίδευσης.

Μεγαλύτερο μέγεθος:

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dense(10)
])


Epoch 10/10
1875/1875 [==============================] - 16s 9ms/step - loss:
0.2226 - accuracy: 0.9164

Test accuracy: 0.8859999775886536
```

Η ακρίβεια του μοντέλου αυξάνεται. Επίσης αυξάνεται ο χρόνος εκπαίδευσης

Δοκιμάζουμε αλλαγή του αριθμού των στρωμάτων

Προσθέτουμε ένα ακόμα στρώμα στο μοντέλο

```
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])
```

```
Epoch 10/10
1875/1875 [==============================] - 9s 5ms/step - loss:
0.2298 - accuracy: 0.9119
```

```
Test accuracy: 0.887499988079071
```

Η ακρίβεια του μοντέλου αυξάνεται. Επίσης αυξάνεται ο χρόνος εκπαίδευσης